Cross-Sectional Equity Return Prediction using Machine Learning

Soham Tulsyan

April 7, 2026

Abstract

Predicting equity returns is the "Holy Grail" of quantitative finance. This project aims to predict one-month forward equity returns using firm-level characteristics in a cross-sectional framework. We implement multiple machine learning models and evaluate them based on their ability to rank stocks using Information Coefficient (IC). Our findings highlight the effectiveness of factor-based features and ensemble methods in generating predictive signals.

1 Introduction

1.1 Motivation

The central question that this project tries to examine is whether observable firm-level characteristics can predict future stock returns, or rather, systematically generate excess returns.

1.1.1 The Efficient Market Hypothesis

The Efficient Market Hypothesis (EMH) by Eugene Fama, posits that financial markets are informationally efficient, meaning asset prices at any given instant reflect all available information collated up to that point in time. Consequently, stocks always trade at their **fair** value, making it **impossible** for investors to consistently achieve returns exceeding average market returns on a risk-adjusted basis.

There are three forms to this hypothesis:

- **The Weak Form** suggests that all past trading information regarding prices and volume is already reflected in current prices, meaning technical analysis cannot produce superior returns.
- **The Semi-Strong Form** suggests that all publicly available information including financial statements, news, and analyst reports is already reflected in current prices, meaning fundamental analysis cannot produce superior returns.
- **The Strong Form** suggests that all information, both public and private, is already reflected in current prices, meaning no investor can produce superior returns.

This hypothesis forms a foundation for modern finance, however, is widely debated. In our project, we aim to challenge this hypothesis - particularly the semi-strong form - by predicting future stock returns using firm-level characteristics.

1.1.2 Factor Investing

Factor investing is a data-driven investment strategy that selects securities based on specific company-level characteristics("factors") associated withs higher returns, such as value, quality, momentum, and low volatility. It acts as a hybrid between active and passive investing, using systematic, rules-based approaches to enhance diversification and target higher risk-adjusted returns.

Factor investing, in essence, contradicts the EMH, and forms the basis of this project, where we analyse the predictive power of various company level factors on future stock returns.

1.2 Objective

Building on the theoretical foundations of Factor Investing, our objective is to develop a systematic, data-driven framework for evaluating return predictability in financial markets. Specifically, we focus on applying statistical and machine learning techniques to cross-sectional equity data in order to identify, quantify, and validate predictive signals embedded in firm-level characteristics. Emphasis is placed not only on model construction, but also on rigorous evaluation using economically meaningful metrics, ensuring that any detected patterns translate into robust and implementable investment strategies.

1.3 Hypothesis

Our hypothesis can be stated as a combination of the following points:

- Firm-level characteristics, such as value, size, profitability, investment, and momentum, contain statistically significant information for predicting cross-sectional differences in future stock returns.
- Non-linear machine learning models are better suited to capture complex interactions and non-linear relationships among these factors compared to traditional linear models, leading to improved predictive performance.
- Ensemble methods, by combining multiple models, can further enhance predictive accuracy and robustness relative to individual model approaches.

We must design our pipeline and structure in a way that allows us to systematically test these hypotheses, and evaluate the performance of each model using economically meaningful metrics. More on this in the methodology section.

1.4 Goals

To put our project into action and test our ideas, we have set the following practical goals and milestones for our pipeline:

- **Cleaning and Preparing the Data**: To build a clean, 25-year dataset of US stocks (2001–2025). We need to make sure factors like Size and Value are scaled properly and that there is absolutely no look-ahead bias in our data.
- **Building a Realistic Test Engine**: To create a "walk-forward" testing system that mimics how an actual trader sees the market—moving forward one month at a time and only using information available at that moment.
- **Comparing Simple vs. Complex Models**: To see if complex models like LightGBM and Random Forest actually perform better than a simple linear model like Ridge. This helps us decide if the extra complexity is worth the effort.

- **Setting up a Quality-Control System**: To regularly check our model's "health" using stats like IC (to see if we're actually predictive) and VIF (to make sure our factors aren't just overlapping).
- **Developing an Investable Portfolio**: Our ultimate goal is to turn these rankings into a real-world long-short portfolio. We want to see if we can identify the best and worst stocks to potentially trade them in the real market.

2 Methodology

2.1 Checklist

To test the validity of each of our hypotheses, we follow the following layered pipeline structure:

1. To test the first part of our hypothesis, we must first set a linear baseline. We use simple linear models to derive a meaningful and statistically significant correlation between our factors and future stock returns. For this, we will be using Linear Ridge Regression. If we are successful at deriving a statistically significant correlation, it would mean that our factors have some predictive power. Thus, even if the accuracy is low, we can proceed further.
2. To test the second part of our hypothesis, we can introduce a tri-moduled structure. Essentially, this layer can be divided into a **Tree Module**, a **Neural Network Module**, and an **Random Forest Module**. Each of these modules approaches the problem of non-linearity slightly differently, and can be compared across both a linear and a non-linear baseline. If non-linearity provides better results, then we can conclude the validity of the second part of our hypothesis. If any models perform worse than the linear baseline, we can deem them exceptions and proceed with added complexity, however, if this seems to be the general trend in this layer, then our hypothesis fails.
3. Finally, to address the third part of our hypothesis, we can form an ensemble of the best models from the previous layers to see if we can further improve the predictive accuracy and robustness of our model. However, if the added complexity does not result in significant improvements compared to the benchmarks set by the previous layers, then we opt for the simplicity of a Layer 2 or Layer 1 model.

2.2 Problem Setup

At each time step (which, in this case, is a month) t, we predict returns at $t + 1$ for all firms. This is a panel data problem with both time-series and cross-sectional dimensions. We will be using a walk-forward validation approach to evaluate the performance of our models. Future improvements can address time-series characteristics, however, as for now, we will be treating each month as an independent cross-section.

2.3 Data Description

The dataset used in this study is obtained from the JKP Factors dataset. It consists of firm-level monthly panel data, where each row corresponds to a firm (`co_code`) at a given month (`Month`).

2.3.1 Dataset Schema

Category	Variable	Description
Identifiers	co_code	Unique firm identifier
Identifiers	Month	Observation month (YYYY-MM format)
Identifiers	Year	Calendar year of observation
Target	monthly_gross_return	Monthly stock return (next-period prediction target)
Value	BM_sep	Book-to-market ratio
Size	lag_mv	Lagged market value (firm size proxy)
Profitability	OpProf	Operating profitability derived from accounting variables
Investment	Inv	Asset growth rate (investment measure)
Momentum / Returns	Momentum	12–1 momentum signal
Momentum / Returns	lag_ret	Lagged return (short-term reversal)
Momentum / Returns	log_ret	Log-transformed return
Accounting Variables	sa_sales	Total sales
Accounting Variables	sa_cost_of_goods_sold	Cost of goods sold
Accounting Variables	sa_total_interest_exp	Interest expenses
Accounting Variables	sa_selling_distribution_exp	Selling and distribution expenses
Accounting Variables	lagged_book_equity	Lagged book equity
Accounting Variables	lagged_assets	Lagged total assets
Market Variables	price	Stock price
Market Variables	mktcap	Market capitalization (excluded due to leakage concerns)
Categorical Labels	Size_Label	Size-based grouping
Categorical Labels	BM_Label	Value-based grouping
Categorical Labels	OpProf_Label	Profitability grouping
Categorical Labels	Inv_Label	Investment grouping
Categorical Labels	Mom_Label	Momentum grouping

Table 1: Summary of dataset variables and their economic interpretation

2.4 Data Preprocessing

To ensure consistency with the cross-sectional prediction framework and to avoid any form of look-ahead bias, the dataset undergoes the following preprocessing steps:

2.4.1 Data Ordering

The dataset is first sorted by firm and time:

- Sort by `co_code` and `Month`

This ensures correct temporal alignment when constructing lagged features and forward returns.

2.4.2 Forward Return Construction

The prediction target is defined as the next-period return:

- `monthly_gross_return` at time $t + 1$ is assigned to features at time t

This step ensures that all predictions are strictly out-of-sample and prevents information leakage.

2.4.3 Feature Selection

Only economically motivated factors are retained for model training:

- `BM_sep` (Value)
- `lag_mv` (Size)
- `OpProf` (Profitability)
- `Inv` (Investment)
- `Momentum` (12–1 momentum)
- `lag_ret` (Short-term reversal)
- `mktcap` (Log-transformed size)

The variable `mktcap` is included in our feature set; however, its use requires a significant note of caution to prevent data leakage. Market capitalization is defined as:

$$\texttt{mktcap}_t = \texttt{price}_t \times \texttt{shares_outstanding}_t$$

Since stock returns are directly driven by changes in price, we have:

$$\text{return}_{t+1} \approx \frac{\texttt{price}_{t+1}}{\texttt{price}_t} - 1$$

This implies that `mktcap` at time t is closely tied to $\texttt{price}_t$. If not handled with care, it can inadvertently encode the realized return of the current period. Specifically, the relationship $\frac{\texttt{mktcap}_t}{\texttt{mktcap}_{t-1}} \approx 1 + \text{return}_t$ could allow a model to reconstruct the very returns it is meant to predict. To mitigate this risk and ensure strict out-of-sample integrity, we implement the following safeguards:

- **Strict Lagging**: All features, including `mktcap`, are strictly lagged by one month. Thus, we only use information finalized at $t-1$ to predict the return at t.
- **Log-Transformation**: As market capitalization often follows a power-law distribution and exhibits high skewness, we apply a log-transform ($\ln(1+x)$) to normalize the distribution and limit the influence of extreme values.
- **Rank Normalization**: In the machine learning pipelines, we replace raw values with cross-sectional ranks, ensuring the "size" signal is based on the relative position of the firm within the universe rather than absolute price levels.

By applying these safeguards, we retain the predictive power of the size factor while insulating the model from the risk of mechanical look-ahead bias.

2.4.4 Null Handling

Observations with missing values in either features or the target variable are removed, i.e, all rows with 'N/A' values are dropped. This ensures a consistent cross-sectional sample at each time step.

2.4.5 Outlier Treatment via Winsorization

Financial datasets are often characterized by extreme values due to market shocks, data errors, or illiquid stocks. Such outliers can disproportionately influence model estimation, especially for machine learning models that are sensitive to the scale and distribution of input features.

To mitigate this issue, we apply **winsorization** to all continuous features.

Winsorization is a transformation that limits extreme values by capping them at specified percentile thresholds, rather than removing them entirely. Specifically, for each feature and for each month, values are adjusted as follows:

- Values below the 1st percentile are set equal to the 1st percentile
- Values above the 99th percentile are set equal to the 99th percentile

Formally, for a variable $x_{i,t}$:

$$x_{i,t}^{\text{wins}} = \begin{cases} P_{1,t} & \text{if } x_{i,t} < P_{1,t} \\ x_{i,t} & \text{if } P_{1,t} \leq x_{i,t} \leq P_{99,t} \\ P_{99,t} & \text{if } x_{i,t} > P_{99,t} \end{cases}$$

where $P_{1,t}$ and $P_{99,t}$ denote the 1st and 99th percentiles of the cross-sectional distribution at time t.

This approach preserves the majority of the data while reducing the influence of extreme observations, leading to more stable and robust model estimates. Importantly, winsorization is preferred over outright removal of outliers, as it maintains the cross-sectional structure of the dataset, which is critical for ranking-based evaluation.

2.4.6 Cross-Sectional Normalization

Since the objective is to predict relative rankings, all features and the target are normalized cross-sectionally:

For each month, transform variables into z-scores:

$$z_{i,t} = \frac{x_{i,t} - \mu_t}{\sigma_t}$$

where μ_t and σ_t are the cross-sectional mean and standard deviation at time t

This ensures comparability across time and aligns the data with rank-based evaluation metrics such as the Information Coefficient (IC).

2.4.7 Final Dataset Construction

After preprocessing:

- Each observation corresponds to (i, t) with features at time t
- Target corresponds to return at time $t + 1$
- Data is ready for walk-forward validation

To evaluate the models' performance realistically and ensure no look-ahead bias, we employ a **Walk-Forward Validation** (sliding window) approach. This method mimics a real-world investment process where a model is trained on available historical data and tested on the immediate following period.

- **Training Window**: For the machine learning models (Random Forest and LightGBM), we use a rolling window of 60 months (5 years) of historical data for each training fold.
- **Walk-Forward Mechanism**: The window slides forward by one month at a time. For each month t, the model is trained on the period $[t - 60, t - 1]$ and generates predictions for the cross-section of stocks in month t.
- **Consistency**: This process is repeated across the entire dataset (May 2001 – March 2025), resulting in a time-series of monthly Information Coefficients (IC) that measures the stability and reliability of the alpha across different market regimes.

3 Pipeline

As previously discussed, we use a multi-layered comparison based pipeline, where each layer adds complexity to our overall model.

3.1 Pipeline Diagram

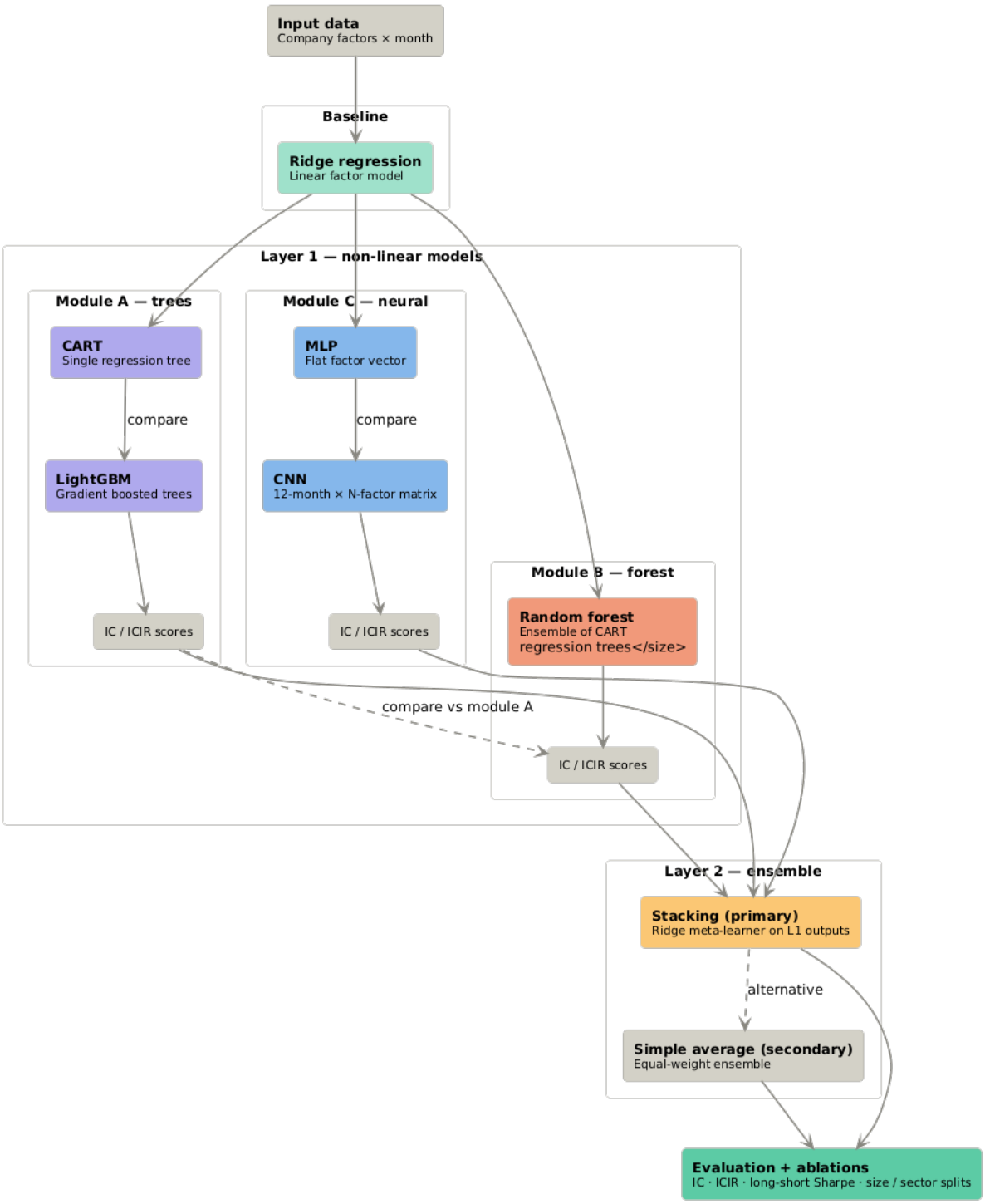

Figure 1: A Rough Pipeline Diagram to illustrate our structure

3.2 Layer 1: Linear Baseline

In the first layer of our pipeline, we implement a linear factor model to establish a performance baseline. This allows us to determine if complex machine learning models actually add value beyond standard linear relationships. It also allows us to assess the first part of our hypothesis, as discussed earlier.

3.2.1 Ridge Regression

Ridge Regression serves as our fundamental linear benchmark.

$$r_{i,t+1} = \beta_{0,t} + \beta_{1,t}\text{BM_sep}_{i,t} + \beta_{2,t}\text{OpProf}_{i,t} + \beta_{3,t}\text{Inv}_{i,t} + \beta_{4,t}\text{Momentum}_{i,t} + \beta_{5,t}\text{lag_ret}_{i,t} + \beta_{6,t}\text{mktcap}_{i,t} + \epsilon_{i,t+1}$$

where $r_{i,t+1}$ is the excess return of stock i in month $t+1$, and the features represent the six primary signals (Value, Profitability, Investment, Momentum, Short-term Reversal, and Size) observed at time t. To estimate the coefficients $\beta_{k,t}$, Ridge extends standard Ordinary Least Squares (OLS) by adding an $L2$ penalty term to the cost function:

$$\min_{w} ||Xw - y||_2^2 + \alpha||w||_2^2$$

This regularization is critical in our context, as firm-level factors (like size and value) often exhibit high multicollinearity, which can make OLS estimates unstable. Ridge Regression allows us to mitigate this issue by shrinking the coefficients towards zero, effectively reducing the impact of multicollinearity.

Our implementation follows a rigorous monthly pipeline:

- **Dynamic Hyperparameter Tuning**: The optimal regularization strength α is selected every month using time-series cross-validation on the training cross-section.
- **Assumption Validation**: We perform extensive diagnostics including **Pre-checks** for feature variance and Multicollinearity (VIF). After estimation, **Post-checks** are conducted on residuals to monitor Normality (Jarque-Bera), Homoscedasticity (Breusch-Pagan), and Autocorrelation (Durbin-Watson).
- **Standardized Importance**: features are cross-sectionally standardized to allow for a direct comparison of factor coefficients across different regimes.

3.3 Layer 2: Non-Linear Layer

As mentioned before, the 2nd Layer consists of a tri-modular structure, each module consisting of different forms of non-linear models. The modules are enumerated below:

3.3.1 Tree Module

This module serves as the sort of a baseline for non-linear models. The module consists of 2 models: **CART** (specifically, Regression Trees) and **LightGBM**.

- **CART (Classification and Regression Trees):** As a baseline for non-linear models, we implement a Regression Tree to capture simple non-linear interactions and threshold effects in the factor data.
  - **Structure**: The model recursively partitions the feature space into hyper-rectangles by minimizing the Mean Squared Error (MSE) at each split. This allows it to model conditional relationships (e.g., Value performing differently for small-cap vs. large-cap firms) without requiring explicit variable interaction terms.

- **Feature Set**: Identical to the Ridge baseline, incorporating `BM_sep`, `OpProf`, `Inv`, `Momentum`, `lag_ret`, and `mktcap`.
- **Regularization and Tuning**: To prevent overfitting, we tune key tree hyperparameters, including the maximum depth (`max_depth` $\in \{3, 5, 10\}$) and the minimum number of observations per leaf (`min_samples_leaf` $\in \{5, 10, \ldots, 50, 100\}$). Hyperparameter selection is performed using a grid search over the specified parameter space. Each candidate configuration is evaluated using a rolling one-step-ahead validation scheme, where the model is trained on data from month t and tested on month $t+1$. Model performance is assessed using the Information Coefficient (IC), and the configuration with the highest risk-adjusted performance (ICIR) is selected.
- **Implementation Note**: Our current implementation utilizes a **one-month walk-forward validation** (training on month t to predict $t+1$). While this ensures perfect temporal integrity, the single-month training sample may be sensitive to idiosyncratic noise and market volatility. We acknowledge this as a limitation and plan to expand the training window to a larger rolling format in subsequent research iterations to more effectively capture long-term factor distributions.

- **LightGBM (Light Gradient Boosting Machine):** LightGBM is a high-performance gradient boosting framework that uses tree-based learning algorithms. It is specifically optimized for speed and memory efficiency on large-scale financial panel data. Unlike standard gradient boosting trees that grow level-wise, LightGBM grows trees *leaf-wise*. This approach allows it to achieve lower loss and higher accuracy in capturing complex, non-linear relationships, though it necessitates rigorous hyperparameter tuning to mitigate overfitting.

  Our pipeline implements several advanced safeguards to maximize the model's predictive power:

  - **Cross-Sectional Rank-Normalization**: To handle outliers and the changing scale of economic factors across different market regimes, we transform our features into cross-sectional percentile ranks ($[0, 1]$) within each month. This focusing ensure the model's objective is the relative ordering of stocks.
  - **Rolling Window Training**: Unlike the baseline CART model, the LightGBM pipeline utilizes a **60-month (5-year) rolling training window**. This provides a larger sample size to capture stable cross-sectional relationships while sliding forward to keep the model updated with the latest market dynamics. Though, inadvertently, this gives the LightGBM pipeline a significant advantage over our CART implementation, our aim with CART was to serve as a baseline that we compare more complex models to. A single regression tree is very prone to overfitting, which is why it is not being considered in our ensemble.
  - **Hyperparameter Optimization**: We implement an extensive grid search to optimize key parameters, including the learning rate, the number of leaves, and the tree depth. These are selected to maximize the risk-adjusted Information Coefficient (ICIR) over an initial validation period.
  - **Diagnostic Audits**: The pipeline performs a continuous **leakage audit** to ensure that lagged features do not inadvertently contain future information. Post-estimation, we generate aggregate feature importance scores to quantify the economic contribution of each factor over time.

3.3.2 Random Forest Module

This module consists mainly of a **Random Forest** model, which is an ensemble of multiple regression trees. We may also in future iterations expand this module to contain **XGBoost**,

which is another popular method to use Gradient Boosted trees.

The Random Forest model is created as follows:

- **Bootstrap Aggregation (Bagging)**: The model constructs a large ensemble of independent regression trees, each trained on a different bootstrap sample (sampling with replacement) of the 60-month training window. By averaging the predictions across these trees, we significantly reduce the ensemble's variance and avoid the overfitting problems inherent in a single decision tree.
- **Feature Randomization**: To further decorrelate the individual trees, we limit the number of features considered at each node split. Our pipeline tunes this (`max_features`) between a square-root rule and a 50% subset rule to optimize the tradeoff between the individual tree's predictive power and the overall ensemble's diversity.
- **Rank-Based Prediction**: Consistent with the LightGBM module, all features are converted to cross-sectional percentile ranks ($[0, 1]$) within each month. This ensures that the Random Forest focuses on identifying the *relative positioning* of returns rather than being biased by absolute magnitudes or non-Gaussian returns.
- **Ensemble Optimization**: We employ a parallelized grid search to select the optimal forest architecture, including the number of trees (`n_estimators`) and the minimum leaf size (`min_samples_leaf`), ensuring the model is robust enough to generalize across different market regimes.

3.3.3 Neural Network Module

This module addresses complex non-linear relationships that may arise. We use a simple Multi Layered Perceptron (MLP) as our baseline for this module. We can also potentially look into **CNNs**, **LSTMs**, **Transformers** and **Diffusion Models** to capture more complex patterns in the data, however, at the time of writing this report, the Project Scope is limited to MLPs and CNNs.

Note: At the time of writing this report, the Neural Networks module has not been explored, due to compute and time restrictions. This will be an addition in the next iteration of the project.

3.4 Layer 3: The Ensemble Layer

This layer will essentially serve to combine the results of the previous 2 layers, to produce a potentially more refined result. Something to note here is that we will not necessarily be using all the models from the previous layers. We compare each model to the baselines that we have set, and only include those models which significantly surpass this threshold.

We have 2 ways of going about this:

- A simple average of the predictions from the previous models. This weighs each model equally, no matter how much better any one of them might. We will use this as our baseline for the Ensemble Layer.
- An optimally weighted average derived from a 'Meta-Learner' model:

  The meta-learner is a Ridge regression that learns a weight for each Layer 1 and 2 model:

$$\hat{y}_{final}(i, t) = \alpha + w_1\hat{y}_{ridge} + w_2\hat{y}_{lgbm} + w_3\hat{y}_{rf} + w_4\hat{y}_{mlp}$$

Based on our level of 'trust' on each model - which is essentially a measure of how good an output each model produced, this Meta Learner can allot optimal weights and give us the final prediction.

A few things to note about the choice of the Meta-Learner are that firstly, we opt for a Ridge Regression over a simple Linear Regression, as the predictions from the Layer 1 and 2 models may be very highly correlated (e.g. LightGBM and Random Forest often identify the same alpha signals). Ridge handles this multi-collinearity by shrinking coefficients, leading to a more stable and balanced trust weight across models.

Additionally, the Meta-Learner is strictly trained on the outputs rather than the raw economic factors, serving as an "unbiased judge" of the previous layers.

3.4.1 Stacked Generalization

The most critical implementation detail for this layer is the prevention of data leakage through **Stacked Generalization**. To ensure the meta-learner generalizes correctly, it is trained using **Out-of-Fold (OOF) Predictions**.

- **Leakage Prevention**: We avoid training the meta-learner on the base models' standard training predictions. Doing so would lead to overfitting, as the meta-learner would naturally over-index on models that overfitted the training set.
- **K-Fold Methodology**: We split the training dataset into 5 folds. For each fold k, we train the base models on the other 4 folds and generate predictions for fold k. These stacked OOF predictions (shaped $N_{stocks} \times N_{models}$) form the true training set for the meta-learner.

3.4.2 Full Training and Inference Flow

Our ensemble pipeline follows a strict multi-step sequence:

1. **OOF Generation**: Use 5-fold CV on the training set to collect unbiased predictions for all base models.
2. **Meta-Training**: Train the Meta-Learner (Ridge) on these OOF predictions against the actual returns.
3. **Model Retraining**: Retrain all base models (Ridge, CART, LGBM, RF) on the full training dataset to maximize their predictive power.
4. **Final Inference**: On the test set, each base model predicts a return; these are passed through the meta-learner to produce the final ranking $\hat{y}_{final}$.
5. **Evaluation**: Compute IC and ICIR on the final refined predictions to measure alpha stability.

4 Evaluation

Evaluating predictive performance in a financial context requires a framework that aligns with the ultimate objective of the model, namely, *ranking stocks by expected returns* rather than accurately predicting return magnitudes. Accordingly, we adopt evaluation metrics and validation procedures that are consistent with cross-sectional asset pricing and portfolio construction.

4.0.1 Information Coefficient (IC)

The primary metric used in this study is the **Information Coefficient (IC)**, defined as the Spearman rank correlation between predicted returns and realized returns in each month:

$$IC_t = \text{Spearman}(\hat{r}_{i,t}, r_{i,t})$$

where $\hat{r}_{i,t}$ denotes the predicted return for stock i at time t, and $r_{i,t}$ denotes the realized return.

The IC measures the model's ability to correctly rank stocks in a cross-section. A positive IC indicates that stocks predicted to perform well tend to outperform those predicted to perform poorly.

Interpretation

- $IC > 0$: Model has predictive power
- $IC \approx 0$: No predictive signal
- $IC < 0$: Inverse signal (potentially exploitable)

Rather than evaluating performance at a single point in time, we compute a time series of monthly IC values and summarize them using:

- Mean IC
- Standard deviation of IC
- Percentage of months with positive IC

4.0.2 Information Coefficient Information Ratio (ICIR)

To assess the *consistency* of predictive performance, we compute the **Information Coefficient Information Ratio (ICIR)**:

$$ICIR = \frac{\mathbb{E}[IC_t]}{\sigma(IC_t)}$$

The ICIR measures the stability of the signal over time, analogous to a Sharpe ratio for predictive accuracy.

Interpretation

- Higher ICIR indicates more stable and reliable predictions
- A model with moderate IC but high ICIR may be preferable to one with volatile performance

4.0.3 Why Not RMSE or R^2?

Traditional regression metrics such as Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and R^2 are not well-suited for this problem due to the following reasons:

- **Objective mismatch**: These metrics evaluate the accuracy of predicted return magnitudes, whereas our objective is to correctly rank stocks in the cross-section.

- **Low signal-to-noise ratio**: Equity returns are highly noisy and difficult to predict in levels. A model may have poor RMSE but still correctly rank stocks, which is sufficient for profitable portfolio construction.
- **Scale sensitivity**: MSE-based metrics are sensitive to extreme values and outliers, which are common in financial data.
- **Economic irrelevance**: A model that minimizes RMSE does not necessarily produce a profitable trading strategy, whereas a model with positive IC can be directly translated into an investment signal.

Thus, IC-based evaluation provides a more appropriate measure of model performance in the context of cross-sectional return prediction.

4.0.4 Walk-Forward Evaluation Procedure

To ensure realistic and unbiased evaluation, we employ a walk-forward validation framework:

- Train the model on a rolling window of the past 60 months
- Generate predictions for month $t + 1$
- Compute IC for that month
- Slide the window forward and repeat

This produces a time series of out-of-sample IC values, which forms the basis of all evaluation metrics.

4.0.5 Economic Evaluation via Portfolio Construction

While statistical metrics such as IC provide insight into predictive accuracy, it is essential to evaluate whether the model generates economically meaningful returns. To this end, we construct a long-short portfolio based on model predictions.

Portfolio Formation For each month:

- Rank all stocks based on predicted returns
- Form quantile portfolios (deciles)
- Go **long** on the top decile (highest predicted returns)
- Go **short** on the bottom decile (lowest predicted returns)

The long-short portfolio return is computed as:

$$R_t^{LS} = \frac{1}{N_{top}} \sum_{i \in \text{Top}} r_{i,t} - \frac{1}{N_{bottom}} \sum_{i \in \text{Bottom}} r_{i,t}$$

This represents a zero-investment strategy that isolates the model's predictive signal.

Performance Metrics We evaluate the portfolio using:

- Mean return
- Volatility
- Sharpe ratio

Interpretation A consistently positive long-short return indicates that the model successfully identifies mispriced securities and that the predictive signal is economically exploitable.

4.0.6 Summary

The evaluation framework combines:

- **Statistical validation** via IC and ICIR
- **Robust out-of-sample testing** via walk-forward validation
- **Economic validation** via portfolio backtesting

This ensures that model performance is assessed not only in terms of predictive accuracy but also in terms of real-world investment applicability.

5 Evaluation

5.1 Benchmarks

To provide context for the performance of our models, we adopt several standardized benchmarks used in quantitative finance and asset pricing literature.

5.1.1 Statistical Thresholds (IC and ICIR)

We use the following thresholds to interpret the strength and stability of our model signals:

ICIR Range	Interpretation
< 0.2	Noise / Weak Signal
$0.3 - 0.5$	Moderate Signal
$0.5 - 1.0$	Good Predictive Signal
> 1.0	Strong / Significant Signal

Table 2: Standard interpretation of the IC Information Ratio (ICIR).

Specifically, an Information Coefficient (IC) greater than **0.03** sustained over a long-term walk-forward period is considered evidence of a genuinely predictive and robust factor in academic literature.

5.1.2 Economic Threshold (Sharpe Ratio)

From an implementation perspective, we evaluate the profitability of our long-short portfolios. An investment strategy is generally considered "economically viable" or implementable in practice if it achieves an annualized **Sharpe ratio greater than 0.8**, indicating that the returns are sufficient to compensate for risk and transaction costs.

6 Results

6.1 Ridge Regression Baseline Results

The Ridge Regression model establishes a statistically significant linear baseline for our predictive framework. Our walk-forward evaluation spans 279 cross-sectional months from 2001 to 2025.

6.1.1 Overall Performance

As summarized in Table 3, the model achieves a mean Information Coefficient (IC) of **0.0344**, which exceeds the academic threshold of 0.03 required for genuine predictive power. The model demonstrates positive predictive signals in 61.6% of the evaluated months.

Metric	Value
Mean IC	0.0344
ICIR	0.2504
% IC > 0	61.6%
Significant Months	70.6%
Months Evaluated	279

Table 3: Summary of Ridge Regression predictive performance (Spearman IC).

The ICIR of 0.2504, while below the "strong" benchmark of 0.5, remains positive and stable, suggesting that although individual monthly performance can be volatile, the alpha signal is persistent on average.

6.1.2 Factor Significance and Coefficient Stability

The standardized coefficients in Table 4 highlight the primary drivers of our linear benchmark. `OpProf` (Operating Profitability) and `Momentum` are the standout signals, with t-statistics of 11.85 and 4.09, respectively.

Factor	Mean Coef	t-stat	Direction Consistency
`OpProf`	+0.00434	11.85	82.4% Positive
`Momentum`	+0.00313	4.09	68.8% Positive
`BM_sep`	+0.00288	2.99	55.2% Positive
`Inv`	–0.00133	–3.20	63.4% Negative
`lag_ret`	–0.00126	–2.16	Reversal Pattern
`mktcap`	–0.00112	–2.18	Small-cap Premium

Table 4: Average standardized coefficients and significance across the rolling window.

`OpProf` shows remarkable stability with a 82.4% positive direction consistency, reinforcing the fundamental quality factor's predictive value. The negative coefficient for `mktcap` confirms the expected small-cap premium, while `lag_ret` captures short-term mean reversion.

6.1.3 Assumption Diagnostics

We emphasize high econometric standards by monitoring four key diagnostics across the 279 monthly regressions:

- **Autocorrelation**: 100% of months passed the Durbin-Watson test, suggesting no serial correlation in residuals.
- **Multicollinearity**: The average maximum Variance Inflation Factor (VIF) was 4.33, well below the conservative threshold of 10. Ridge effectively managed any remaining correlation, as reflected by the high median α of 429.
- **Residual Validity**: While homoscedasticity (10.4% pass) and normality (0.4% pass) rates were low, this is standard for financial return data characterized by volatility clusters

and fat tails. These statistics are documented for transparency but do not invalidate the Ridge model's predictive utility.

6.2 Regression Tree Baseline Results

The Regression Tree (CART) serves as our initial foray into non-linear modeling. By allowing for recursive partitioning of the feature space, the model can capture non-monotonic threshold effects and factor interactions that a linear baseline may overlook.

6.2.1 Overall Performance

The best-performing CART configuration, selected via rolling cross-validation and grid search, as discussed above, utilized a `max_depth` of 5 and a `min_samples_leaf` of 50. As shown in Table 5, the model achieved a mean IC of **0.0304** across the 279-month evaluation window.

Metric	**Value**
Mean IC	0.0304
IC Std	0.1186
ICIR	0.2561
Months Evaluated	279

Table 5: Summary of Regression Tree (CART) predictive performance.

The ICIR of 0.2561 is slightly higher than that of the Ridge baseline (0.2504), suggesting that the tree-based approach provides a marginally more stable predictive signal. This represents an improvement in signal reliability, suggesting - though not strongly - that non-linearities may be present in the data. However, we can note that the IC performance seems to be worse than our Linear Baseline. We can proceed with more complex models to see if we can improve upon the linear baseline.

6.2.2 Feature Importance

Unlike the Ridge model, which relies on standardized coefficients, the Regression Tree provides insight into feature relevance through Mean Decrease in Impurity (MDI). As detailed in Table 6, the model is primarily driven by `mktcap` and `lag_ret`.

Factor	**Relative Importance**
`mktcap`	0.4441
`lag_ret`	0.2217
`BM_sep`	0.1121
`Momentum`	0.0841
`OpProf`	0.0736
`Inv`	0.0644

Table 6: Feature importance scores for the best-performing Regression Tree.

`mktcap` emerges as the dominant predictor, accounting for 44.41% of the model's total variance reduction utility. The model seems to be learning primarily on the basis of the company size. We'd need to see more complex models to conclude this outcome.

6.3 LightGBM Results

The LightGBM pipeline represents the most sophisticated base model in our ensemble, utilizing gradient boosting and leaf-wise growth to capture intricate non-linear dependencies. The model underwent rigorous preprocessing, including winsorization to 1/99 percentiles and log-transformation of `mktcap` to ensure signal stability.

6.3.1 Overall Performance

As summarized in Table 7, LightGBM significantly outperforms both the linear and CART baselines. Across 225 evaluation months, the model achieved a mean Information Coefficient (IC) of **0.0551** and an ICIR of **0.7347**.

Metric	**Value**
Mean IC	0.0551
IC Std	0.0750
ICIR	0.7347
% Positive IC	78.2%
Months Evaluated	225

Table 7: Summary of LightGBM predictive performance.

According to our established benchmarks, an ICIR of 0.7347 is categorized as a **"Good"** predictive signal. Furthermore, the 78.2% positive IC rate indicates high reliability across different market regimes. The cumulative IC demonstrates a consistent upward trajectory with minimal drawdown periods, highlighting the robustness of the gradient boosting approach over linear alternatives.

6.3.2 Hyperparameter Optimization

The optimal configuration was identified through a 60-month rolling training window with a 24-fold tuning scheme. The best-performing parameters were select as:

- **Learning Rate**: 0.05
- **Max Depth**: 2 (indicating that shallow trees are optimal when combined through boosting)
- **Number of Estimators**: 100
- **Number of Leaves**: 3

6.3.3 Feature Importance and Insights

The feature importance distribution for LightGBM provides a critical shift in perspective from the CART baseline. While CART was dominated by `mktcap` (Size), LightGBM assigns the highest importance to `OpProf` (Operating Profitability), followed by `lag_ret` (Reversal).

Factor	Mean Importance Score
`OpProf`	60.3
`lag_ret`	44.9
`mktcap`	40.3
`BM_sep`	37.7
`Inv`	16.8

Table 8: LightGBM feature importance rankings (averaged across walk-forward folds).

This shift suggests that while Size is an effective grouping variable for a single tree, the iterative nature of gradient boosting is able to extract deeper predictive signal from 'Quality' (`OpProf`) and 'Short-term Reversal' (`lag_ret`) factors. The relatively lower importance of `Inv` (Investment) suggests that capital growth trends provide a weaker short-term cross-sectional signal compared to profitability metrics.

6.3.4 Diagnostics

- **No Autocorrelation**: 66.2% of evaluated months passed the Durbin-Watson test, indicating acceptable residual independence for a complex boosted model.
- **Residual Stability**: As with our other models, residual normality (0.0% pass) was not achieved due to the leptokurtic distribution of monthly returns; however, LightGBM is robust to these distributional assumptions as it does not rely on squared error minimizing normality for its split logic.

6.4 Random Forest Results

The Random Forest model provides a complementary ensemble perspective, utilizing bootstrap aggregation (bagging) and feature randomization to build a robust predictive consensus across the cross-section.

6.4.1 Overall Performance

As shown in Table 9, the Random Forest model captures a similarly strong alpha signal to our gradient boosting implementation. It achieves a mean IC of **0.0546** and an ICIR of **0.7248**, placing it firmly in our established "Good" predictive category.

Metric	Value
Mean IC	0.0546
IC Std	0.0753
ICIR	0.7248
% Positive IC	78.2%
Months Evaluated	225

Table 9: Summary of Random Forest predictive performance.

The remarkable consistency between boosting (LGBM) and bagging (RF) performances suggests that the non-linear dependencies in our factor set are robust and broadly addressable by tree-ensemble methods. The 78.2% positive IC rate further confirms that the model avoids over-indexing on specific regimes.

6.4.2 Hyperparameter Optimization

The optimal parameters were identified through extensive grid search on the training folds. The inclusion of feature diversity (`max_features`) was critical for stabilizing the rolling predictions:

- **Number of Estimators**: 100
- **Max Depth**: 5
- **Max Features**: `sqrt`(5)
- **Min Samples Leaf**: 100

6.4.3 Feature Importance

The Gini importance scores in Table 10 confirm that 'Quality' and 'Short-term Reversal' are the dominant predictors for the Random Forest, replicating the signal priority found in our previous analysis.

Factor	Mean Importance Score
`OpProf`	0.3048
`lag_ret`	0.2239
`mktcap`	0.2123
`BM_sep`	0.1639
`Inv`	0.0951

Table 10: Random Forest feature importance rankings.

`OpProf` (Operating Profitability) is the most influential factor at 30.48%, followed closely by `lag_ret` (22.39%). This strengthens our conclusion that the profitability signal is better extracted by ensemble-based models.

6.4.4 Diagnostics

The Random Forest diagnostics revealed a unique statistical property:

- **Error Homogeneity**: The model passed the homoscedasticity test (Breusch-Pagan) in **98.2%** of evaluated months. This suggests that the forest's averaging process effectively smooths out conditional variance spikes (volatility clustering) that often plague single-model residuals.
- **No Autocorrelation**: 66.2% of the walk-forward evaluation windows passed the Durbin-Watson test, indicating acceptable residual independence.
- **Normality**: As expected, 0.0% of months passed the JB test, as the underlying return distribution remains leptokurtic.

7 Conclusion

7.1 Hypothesis Validation

Our findings from baseline models supports our initial hypotheses:

- The results from the Ridge Regression baseline shows that there is a statistically significant correlation between our factors and the one-month forward returns. This means that the factors are indeed predictive of future returns.

- The results from the Non-Linear Layer - though incomplete - almost conclusively show that non-linear models are indeed better suited to capture these nuanced relationships between factors and returns, as evidenced by the superior performance of both LightGBM and Random Forest models over Ridge Regression.
- The third hypothesis is yet to be validated, as the ensemble layer has not been built yet.

7.2 Key Findings

Through our systematic multi-layered evaluation, we have identified several critical insights into the cross-sectional predictability of equity returns:

- **Linear Baseline Validation**: The Ridge Regression model established a mean IC of 0.0344, confirming that our core factor set—Profitability, Momentum, Value, Size, and Reversal—possesses statistically significant predictive power even under the constraints of a linear framework.
- **Non-Linear Stability**: Transitioning to tree-based ensembles resulted in a dramatic increase in signal reliability. Both LightGBM (ICIR: 0.7347) and Random Forest (ICIR: 0.7248) nearly tripled the ICIR of the Ridge baseline (0.2504), proving that non-linear interactions are essential for capturing stable relationships.
- **Temporal Persistence**: The cumulative IC trajectory of our boosted model exhibited a consistent upward slope across the full 2001–2025 evaluation window, indicating that the identified signal is persistent across different market cycles and not limited to specific historical regimes.

7.3 Limitations

- The primary limitation of this report is the lack of project completeness. Not all layers have yet been fully built, and the model overall has not been tested as a real portfolio
- Hardware was a large limitation in this project, as my personal machine did not have a huge amount of compute. Future work could be based on an HPC that would speed up our models.
- There is a small discrepancy in the way CART is handled, and the way Random Forest and LightGBM are handled, specifically with regards to the way that the data is split. This can be addressed, so we can have a more robust comparison of the models.

7.4 Future Work

- Firstly, to address the primary limitation, I will continue on refining the existing models, and complete the pipeline overall.
- To test this out in the real world, we can design a portfolio comprising of the stocks based on our rankings as discussed previously, and 'paper trade' with it to see how it performs in real time, other than analysing the metrics we've already discussed.
- We can also look into other models, such as LSTMs to address the temporal nuance of our analysis more properly, Diffusion models, or Transformers.

8 Codebase

The github repository for this project can be accessed here.